

Analyse Technique Profonde : Backend GeoMeteo (app.py)

Ce document dissèque le fichier `app.py` bloc par bloc pour en expliquer la logique modulaire, les choix de résilience et les optimisations spécifiques au déploiement en environnement contraint (Cloud Free Tier).

1. Architecture Modulaire & Immuabilité

```
from algo import PortfolioBrain # IMPORT DU CERVEAU
```

```
@dataclass(frozen=True)
class Settings:
    api_key: str
    openweather_url: str =
    "[https://api.openweathermap.org/data/2.5/weather](https://api.openweathermap.org/data/2.5
    /weather)"
    retry_total: int = int(os.getenv("RETRY_TOTAL", "2"))
```

Analyse Technique :

- **Séparation des Préoccupations (SoC)** : L'import de `PortfolioBrain` montre que la logique algorithmique pure est isolée du serveur web. On peut modifier "le cerveau" sans risquer de casser la gestion des routes.
- **`dataclass(frozen=True)`** : Nous utilisons une classe de données "gelée". Une fois les paramètres chargés au démarrage, ils ne peuvent plus être modifiés accidentellement.
- **Gestion des Secrets (`os.getenv`)** : Respect de la méthodologie *12-Factor App*. Aucun secret n'est écrit en dur, prévenant les fuites sur GitHub.

Pourquoi ce choix ?

Cela garantit une **robustesse totale**. Si une fonction essayait de modifier l'URL de l'API ou la clé de sécurité par erreur, Python lèverait une exception. C'est une protection contre les "effets de bord".

2. Observabilité & Journalisation Structurée

```
logging.basicConfig(format="%(message)s")
```

```
@app.before_request
def _before_request():
```

```
g.request_id = request.headers.get("X-Request-Id") or uuid.uuid4().hex[:12]
g.start_time = time.time()
```

Analyse Technique :

- **Logs "JSON-like"** : Au lieu d'un simple `print()`, nous utilisons une journalisation structurée que des outils comme Datadog ou la console Render peuvent parser facilement.
- **Traçabilité (`request_id`)** : Chaque requête reçoit un ID unique pour corréler les logs d'un même utilisateur.

Pourquoi ce choix ?

En production "headless" (sans écran), les logs sont nos yeux. Le calcul du temps (`dur_ms`) permet de détecter un ralentissement du serveur avant même que les utilisateurs ne s'en plaignent.

3. Limiteur de Débit (Rate Limiting)

```
limiter = Limiter(..., storage_uri="memory://")
```

Analyse Technique :

- **Protection DoS** : Empêche un script malveillant de saturer le CPU ou d'épuiser le quota OpenWeather.
- **Stratégie `memory://`** : Utilisation de la RAM du processus plutôt qu'une base Redis externe.

Pourquoi ce choix ?

C'est un choix **pragmatique** pour le Free Tier. Puisque nous n'avons qu'un seul serveur, le partage de données entre instances n'est pas nécessaire. Cela évite d'ajouter un coût de base de données supplémentaire tout en protégeant l'infrastructure.

4. Réseau Résilient (Retry & Keep-Alive)

```
session = requests.Session()
retry = Retry(
    total=settings.retry_total,
    backoff_factor=settings.retry_backoff_factor,
    status_forcelist=(429, 500, 502, 503, 504),
)
```

Analyse Technique :

- **Session() (Keep-Alive)** : On maintient la connexion TCP ouverte. On évite de refaire le "handshake" TLS à chaque appel, ce qui divise par deux la latence réseau.
- **Exponential Backoff** : En cas d'échec, le code attend (0.4s, puis 0.8s, etc.) avant de réessayer.

✓ Pourquoi ce choix ?

Internet n'est pas fiable. Ce bloc permet à l'application de "guérir" toute seule des micro-coupures réseau sans renvoyer d'erreur 500 à l'utilisateur.

5. Algorithme Jacobson/Karn (Le Cerveau)

```
dynamic_timeout = brain.get_timeout()
try:
    resp = session.get(..., timeout=(3, dynamic_timeout))
    brain.update(latency, success=True)
except requests.Timeout:
    brain.update(latency, success=False)
```

🔍 Analyse Technique :

- **Timeout Dynamique** : Contrairement aux timeouts statiques de 10s, nous calculons le délai "juste" basé sur la congestion réelle.
- **Punition de Karn** : En cas de timeout, on ignore la mesure pour ne pas fausser la moyenne et on augmente le délai futur.

✓ Pourquoi ce choix ?

C'est l'innovation majeure. L'application s'adapte à la météo du réseau comme elle s'adapte à la météo réelle. Cela évite de bloquer les ressources du serveur inutilement.

6. Gestion du Cache & Éviction de Mémoire

```
if len(_cache) >= settings.cache_max_items:
    for i, k in enumerate(list(_cache.keys())):
        _cache.pop(k, None)
        if i >= settings.cache_max_items // 10: break
```

🔍 Analyse Technique :

- **TTL (Time To Live)** : Données périmées après 120s pour garantir la fraîcheur.
- **Éviction Manuelle** : Suppression de 10% des éléments les plus anciens si le cache est plein.

✓ Pourquoi ce choix ?

Sur Render, nous n'avons que **512 Mo de RAM**. Ce mécanisme prévient les fuites de mémoire (*Memory Leaks*) et garantit que le serveur ne crashera jamais par manque d'espace, même après des mois d'activité.

7. Sécurité : Pattern Proxy & Hardening

```
@app.after_request
def _after_request(response):
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-Content-Type-Options"] = "nosniff"
```

Analyse Technique :

- **Pattern Proxy** : Le client JS ne parle qu'à notre backend. La **API_KEY** ne quitte jamais le serveur.
- **Security Headers** : **DENY** empêche le Clickjacking. **nosniff** empêche le navigateur d'exécuter du code malveillant déguisé en texte.

Pourquoi ce choix ?

La sécurité ne doit pas dépendre de bibliothèques lourdes. Ici, nous appliquons les standards OWASP directement dans le cycle de vie de la requête Flask.

8. Mapping Sémantique des Erreurs

```
if resp.status_code == 429:
    return jsonify({"error": "Service surchargé"}), 503
```

Analyse Technique :

- Traduction des codes erreurs du fournisseur (OpenWeather) vers des codes sémantiques pour notre client. Une surcharge externe (429) devient une indisponibilité temporaire (503) pour nous.

Pourquoi ce choix ?

Cela permet au client frontend de savoir exactement comment réagir (réessayer plus tard ou corriger la saisie) sans exposer la logique interne du fournisseur tiers.

Document conçu pour certifier la qualité industrielle du backend GeoMeteo.